



National University of Sciences and Technology (NUST)
School of Electrical Engineering and Computer Science

Department of Computing

CS220: Database Systems

Class: BSAI – 2A

Lab 03: Querying Relational Database

CLO-2: Formulate SQL queries to retrieve information from a relational database.

CLO 3: Build SQL (DDL &DML) queries to retrieve information from a relational database.

Date: 12 February 2026

Time: 10:00 - 13:00

Instructor: Sara Tariq Sheikh

Lab engineer: Saood Sarwar



Introduction

Structured Query Language (SQL) is a high level query language which has inbuilt operators and functions for different presentation/manipulation of the data that is retrieved.

Objectives

After performing this lab students should be able to:

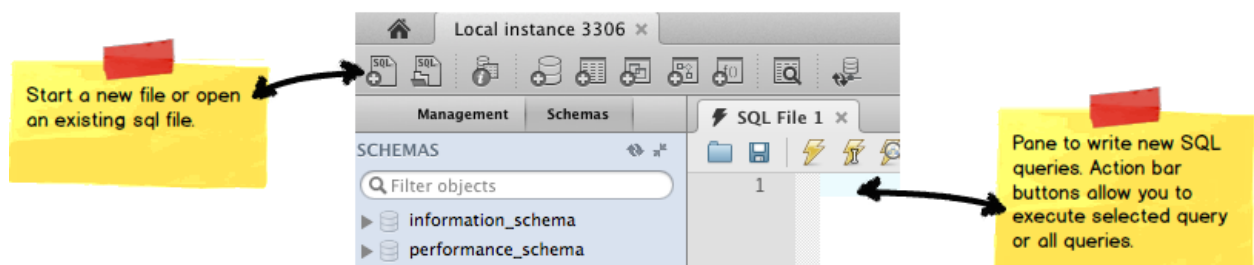
1. Design SQL queries to retrieve data using SELECT clause and using logical operators, SQL operator precedence.
2. Explore and learn various inbuilt single row functions of SQL.

Tools/Software Requirement

- MySQL Community Server
- MySQL Workbench
- Sakila Database

Description

1. Open MySQL Workbench and open the default connection instance.
2. A new query window would open from where you can write and execute queries.



3. You can save the query file and can also add comments using `//`, `/* */` symbols.
4. On executing queries, results are displayed in the lower part of the screen.
5. Error or success messages are displayed in action output pane at the bottom.
6. Continue playing with the Workbench and SQL queries till you are comfortable with the querying mechanism and have learnt the shortcuts to execute queries.

This lab is about querying databases. This lab will cover SQL keywords, functions and logical operators. You will execute these queries using Sakila database. Modify the examples wrt Sakila database and practice all operators/functions on the data in Sakila.



ORDER BY CLAUSE

The SQL ORDER BY clause allows you to sort the records in your result set. The SQL ORDER BY clause can only be used in [SQL SELECT statements](#).

The syntax for the SQL ORDER BY clause is:

```
SELECT columns  
FROM tables  
WHERE predicates  
ORDER BY column ASC/DESC;
```

The SQL ORDER BY clause sorts the result set based on the columns specified. If the ASC or DESC value is omitted, it is sorted by ASC.

ASC indicates ascending order. (default)

DESC indicates descending order.

Example 1:

```
SELECT supplier_city  
FROM suppliers  
WHERE supplier_name = 'IBM'  
ORDER BY supplier_city;
```

Example 2:

When sorting your result set in descending order, you use the DESC attribute in your ORDER BY clause as follows:

```
SELECT supplier_city  
FROM suppliers  
WHERE supplier_name = 'IBM'  
ORDER BY supplier_city DESC;
```

This SQL ORDER BY example would return all records sorted by the supplier_city field in descending order.

SQL ORDER BY - Using both ASC and DESC attributes together

When sorting your result set using the SQL ORDER BY clause, you can use the ASC and DESC attributes in a single [SQL SELECT statement](#).

For example:

```
SELECT supplier_city, supplier_state  
FROM suppliers  
WHERE supplier_name = 'IBM'  
ORDER BY supplier_city DESC, supplier_state ASC;
```

This SQL ORDER BY would return all records sorted by the supplier_city field in descending order, with a secondary sort by supplier_state in ascending order.



SQL Operator Precedence

1. Parentheses - if you group conditional SQL statements together by parentheses, SQL Server evaluates the contents of these first.
2. Arithmetic - multiplication (using the operators *, /, or %)
3. Arithmetic - addition (using the operators + or -)
4. Other - String Concatenate (+)
5. Logical - NOT
6. Logical - AND
7. Logical - OR

Note: (The order of evaluation at the same precedence level is from left to right.)

1. Execute the following queries using the Sakila schema.

```
Select customer_id, 100*amount-10  
from payment;
```

```
Select customer_id, 100*(amount-10)  
from payment;
```

Logical - AND

The AND condition allows you to create an SQL statement based on 2 or more conditions being met. It can be used in any valid SQL statement - select, insert, update, or delete.

The syntax for the AND condition is:

```
SELECT columns  
FROM tables
```

```
WHERE column1 = 'value1'  
AND column2 = 'value2';
```

The AND condition requires that each condition be must be met for the record to be included in the result set. In this case, column1 has to equal 'value1' and column2 has to equal 'value2'.

Example #1:

The first example that we'll take a look at involves a very simple example using the AND condition.



National University of Sciences and Technology (NUST) School of Electrical Engineering and Computer Science

```
SELECT *  
FROM suppliers  
WHERE city = 'New York'  
and type = 'PC Manufacturer';
```

This would return all suppliers that reside in New York and are PC Manufacturers. Because the * is used in the select, all fields from the supplier table would appear in the result set.

Logical - OR

The OR condition allows you to create an SQL statement where records are returned when any one of the conditions are met. It can be used in any valid SQL statement - select, insert, update, or delete.

The syntax for the OR condition is:

```
SELECT columns  
FROM tables  
WHERE column1 = 'value1'  
or column2 = 'value2';
```

The OR condition requires that any of the conditions must be met for the record to be included in the result set. In this case, column1 has to equal 'value1' OR column2 has to equal 'value2'.

Example #1:

The first example that we'll take a look at involves a very simple example using the OR condition.

```
SELECT *  
FROM suppliers  
  
WHERE city = 'New York'  
or city = 'Newark';
```

This would return all suppliers that reside in either New York or Newark. Because the * is used in the select, all fields from the suppliers table would appear in the result set.

Example #2:

The next example takes a look at three conditions. If any of these conditions is met, the record will be included in the result set.

```
SELECT supplier_id  
FROM suppliers  
WHERE name = 'IBM'  
or name = 'Hewlett Packard'  
or name = 'Gateway';
```



This SQL statement would return all `supplier_id` values where the supplier's name is either IBM, Hewlett Packard or Gateway.

Logical Operator Precedence

Logical operators are executed in the following specified order.

1. Logical - NOT
2. Logical - AND
3. Logical - OR

Combination of And & Or

The AND and OR conditions can be combined in a single SQL statement. It can be used in any valid SQL statement - select, insert, update, or delete.

When combining these conditions, it is important to use brackets so that the database knows what order to evaluate each condition.

Example #1:

The first example that we'll take a look at an example that combines the AND and OR conditions.

```
SELECT *  
FROM suppliers  
WHERE (city = 'New York' and name = 'IBM')  
or (city = 'Newark');
```

This would return all suppliers that reside in New York whose name is IBM and all suppliers that reside in Newark. The brackets determine what order the AND and OR conditions are evaluated in.

Example #2:

The next example takes a look at a more complex statement.

```
SELECT supplier_id  
FROM suppliers  
WHERE (name = 'IBM')  
or (name = 'Hewlett Packard' and city = 'Atlantic City')  
or (name = 'Gateway' and status = 'Active' and city = 'Burma');
```

This SQL statement would return all `supplier_id` values where the supplier's name is IBM or the name is Hewlett Packard and the city is Atlantic City or the name is Gateway, the status is Active, and the city is Burma.



Single Row Functions

What is a function?

A function is similar to an operator in operation. A function is a name that performs a specific task. A function may or may not take values (arguments) but it always returns a value as the result. If function takes values then these values are to be given within parentheses after the function name. The following is the general format of a function.

function [(argument-1, argument-2,...)]

If the function doesn't take any value, then function name can be used alone and even parentheses are not required.

Single-row functions return a single result row for every row of a queried table or view. These functions can appear in select lists, WHERE clauses, START WITH and CONNECT BY clauses, and HAVING clauses.

Arithmetic functions take numeric data; date functions take date-type data and string functions take strings. Conversion functions are used to convert the given value from one type to another.

Miscellaneous functions perform operations on any type of data. Group functions are used to perform operations on the groups created by GROUP BY clause.

Character Functions

Character functions operate on values of datatype CHAR or VARCHAR.

LOWER

Returns a given string in lower case.

```
select LOWER(first_name)
from actor;
```

UPPER

Returns a given string in UPPER case.

```
select UPPER(first_name)
from actor;
```

LENGTH

Returns the length of a given string.

```
select length(first_name)
from actor;
```



CONCAT(str1,str2,...)

Returns the string that results from concatenating the arguments. May have one or more arguments.

```
SELECT CONCAT('My', 'S', 'QL');
```

```
+-----+
| CONCAT('My', 'S', 'QL') |
+-----+
```

MySQL

CONCAT_WS(separator,str1,str2,...)

CONCAT_WS() stands for Concatenate With Separator and is a special form of CONCAT(). The first argument is the separator for the rest of the arguments. The separator is added between the strings to be concatenated. The separator can be a string, as can the rest of the arguments. If the separator is NULL, the result is NULL.

```
SELECT CONCAT_WS(',', 'First name', 'Last Name');
```

```
+-----+
| CONCAT_WS(',', 'First name', 'Last Name') |
+-----+
| First name, Last Name |
+-----+
```

LPAD(str,len,padstr)

Returns the string str, left-padded with the string padstr to a length of len characters. If str is longer than len, the return value is shortened to len characters.

```
SELECT LPAD('hi',4,'??');
```

```
+-----+
| LPAD('hi',4,'??') |
+-----+
| ??hi |
+-----+
```

RPAD(str,len,padstr)

Returns the string str, right-padded with the string padstr to a length of len characters. If str is longer than len, the return value is shortened to len characters.

```
SELECT RPAD('hi',5,'?');
```

```
+-----+
```




```
| RPAD('hi',5,'?') |
+-----+
| hi??? |
+-----+
```

LTRIM(str)

Returns the string str with leading space characters removed.

```
SELECT LTRIM(' lecture');
+-----+
| LTRIM(' lecture') |
+-----+
| lecture           |
+-----+
```

REPEAT(str,count)

Returns a string consisting of the string str repeated count times. If count is less than 1, returns an empty string. Returns NULL if str or count are NULL.

```
SELECT REPEAT('MySQL', 3);
+-----+
| REPEAT('MySQL', 3) |
+-----+
| MySQLMySQLMySQL    |
+-----+
```

RTRIM(str)

Returns the string str with trailing space characters removed.

```
SELECT RTRIM('barbar ');
+-----+
| RTRIM('barbar ') |
+-----+
| barbar           |
+-----+
```

SUBSTRING(str,pos)



SUBSTRING(str FROM pos)

SUBSTRING(str,pos,len)

SUBSTRING(str FROM pos FOR len)

The forms without a len argument return a substring from string str starting at position pos. The forms with a len argument return a substring len characters long from string str, starting at position pos. The forms that use FROM are standard SQL syntax. It is also possible to use a negative value for pos. In this case, the beginning of the substring is pos characters from the end of the string, rather than the beginning. A negative value may be used for pos in any of the forms of this function.

```
SELECT SUBSTRING('Quadratically',5);
```

```
+-----+
| SSUBSTRING('Quadratically',5) |
+-----+
| ratically                    |
+-----+
```

```
> SELECT SUBSTRING('foobarbar' FROM 4);
```

```
+-----+
| SUBSTRING('foobarbar' FROM 4) |
+-----+
| barbar                        |
+-----+
```

```
> SELECT SUBSTRING('Quadratically',5,6);
```

```
+-----+
| SUBSTRING('Quadratically',5,6) |
+-----+
| ratica                         |
+-----+
```

SUBSTRING_INDEX(str,delim,count)

Returns the substring from string str before count occurrences of the delimiter delim. If count is positive, everything to the left of the final delimiter (counting from the left) is returned. If count is negative, everything to the right of the final delimiter (counting from the right) is returned. SUBSTRING_INDEX() performs a case-sensitive match when searching for delim.

```
SELECT SUBSTRING_INDEX('www.mysql.com', '.', 2);
```

```
+-----+
```



```
| SUBSTRING_INDEX('www.mysql.com', '.', 2) |  
+-----+  
| www.mysql |  
+-----+
```

Arithmetic Operators:

Arithmetic operators and functions are available in MYSQL which are used for arithmetic operations. You can explore them online.



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

LAB TASKS

Question No. 1: Select the names of actors whose IDs are between 50 and 150, or those whose last name starts with A.

CODE:

```
1 • Select first_name, last_name
2 From actor
3 Where actor_id Between 50 and 150
4 OR last_name LIKE "A%"
```

OUTPUT:

Result Grid			Filter Rows:
	first_name	last_name	
▶	DEBBIE	AKROYD	
	MERYL	ALLEN	
	NATALIE	HOPKINS	
	GARY	PHOENIX	
	CARMEN	HUNT	
	MENA	TEMPLE	
	PENELOPE	PINKETT	
	FAY	KILMER	
	DAN	HARRIS	
	JUDE	CRUISE	
	CHRISTIAN	AKROYD	
	DUSTIN	TAUTOU	
	HENRY	BERRY	



National University of Sciences and Technology (NUST) School of Electrical Engineering and Computer Science

Question No. 2: Write a query to display the names of customers in the following format.

SMITH, M

JOHNSON, P

WILLIAMS, L

JONES, B

BROWN, E

DAVIS, J

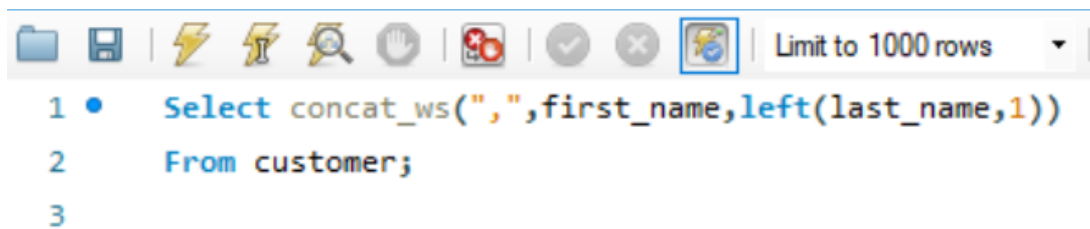
MILLER, M

WILSON, S

MOORE, M

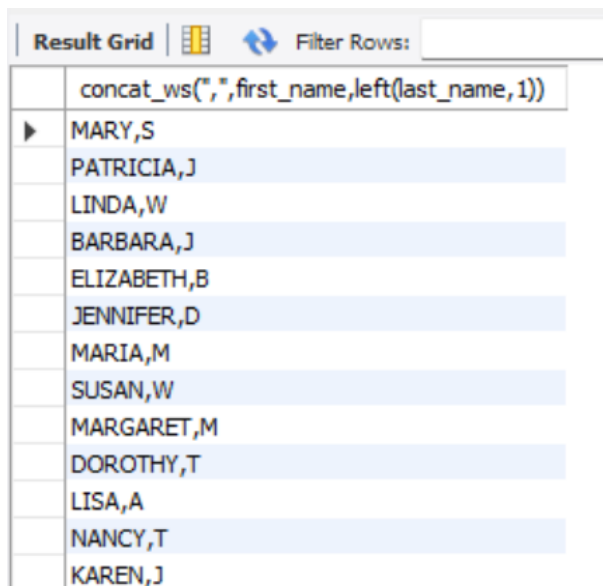
TAYLOR, D

CODE:



```
1 • Select concat_ws(", ", first_name, left(last_name, 1))
2   From customer;
3
```

OUTPUT:



	concat_ws(", ", first_name, left(last_name, 1))
▶	MARY,S
	PATRICIA,J
	LINDA,W
	BARBARA,J
	ELIZABETH,B
	JENNIFER,D
	MARIA,M
	SUSAN,W
	MARGARET,M
	DOROTHY,T
	LISA,A
	NANCY,T
	KAREN,J



National University of Sciences and Technology (NUST) School of Electrical Engineering and Computer Science

Question No. 3: Retrieve the information showing the details of each of the Films released upto now in the format: **Film Academy Dinosaur was released in the year 2006**.

CODE:

```
1 • Select concat_ws(" ", "Film", title, "was released in the year", release_year)
2     From film;
3
```

OUTPUT:

	concat_ws(" ", "Film", title, "was released in the year", release_year)
▶	Film ACADEMY DINOSAUR was released in the year 2006
	Film ACE GOLDFINGER was released in the year 2006
	Film ADAPTATION HOLES was released in the year 2006
	Film AFFAIR PREJUDICE was released in the year 2006
	Film AFRICAN EGG was released in the year 2006
	Film AGENT TRUMAN was released in the year 2006
	Film AIRPLANE SIERRA was released in the year 2006
	Film AIRPORT POLLOCK was released in the year 2006
	Film ALABAMA DEVIL was released in the year 2006
	Film ALADDIN CALENDAR was released in the year 2006
	Film ALAMO VIDEOTAPE was released in the year 2006
	Film ALASKA PHANTOM was released in the year 2006
	Film ALI FOREVER was released in the year 2006



National University of Sciences and Technology (NUST) School of Electrical Engineering and Computer Science

Question No. 4: Display the usernames and address_ids of customers in the Sakila database.

CODE:

```
1 • Select concat(first_name,last_name),address_id
2   from customer
```

OUTPUT:

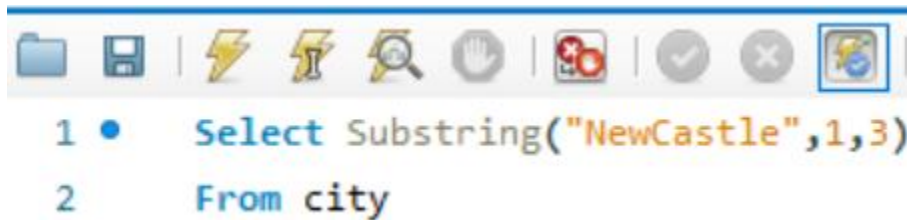
	concat(first_name,last_name)	address_id
►	MARYSMITH	5
	PATRICIAJOHNSON	6
	LINDAWILLIAMS	7
	BARBARAJONES	8
	ELIZABETHBROWN	9
	JENNIFERDAVIS	10
	MARIAMILLER	11
	SUSANWILSON	12
	MARGARETMOORE	13
	DOROTHYTAYLOR	14



Question No. 5: Write two queries using the Substring functions on sakila database.

CODE:

1-



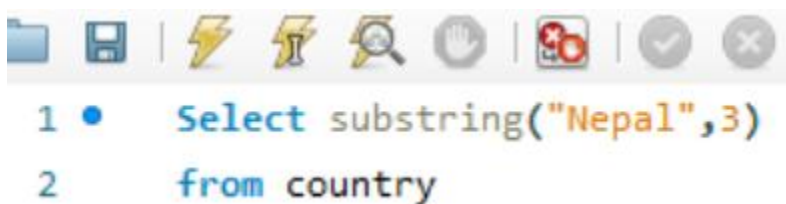
```
1 • Select Substring("NewCastle",1,3)
2 From city
```

Here I used:

SUBSTRING(string, start_position, length):

1. **String:** "NewCastle"
2. **Start (1):** It begins at the very first letter, 'N'.
3. **Length (3):** It counts three

2-



```
1 • Select substring("Nepal",3)
2 from country
```

Here I used:

SUBSTRING (string, position)

1. **String:** "Nepal"
2. **Start (3):** It begins at the 3rd letter, 'P'.
3. **End (5):** It counts from p till the end



OUTPUT:

1-

	Substring("NewCastle",1,3)
▶	New
	New
	New
	New
	New
	New
	New
	New
	New
	New
	New
	New
	New
	New
	New

2-



National University of Sciences and Technology (NUST) School of Electrical Engineering and Computer Science

	substring("Nepal",3)
▶	pal
	pal
	pal
	pal
	pal
	pal
	pal
	pal

Deliverable

Submit a word document including the SQL queries to answer above-mentioned information needs as well as snapshot of their outcome when executed over MySQL using the Workbench.